

Practical – Week10

1. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
ajithmutyala@Ajith:~$ echo "Hello Linux Inodes" > file1.txt
ajithmutyala@Ajith:~$ ls -li file1.txt
40714 file1.txt
```

```
ajithmutyala@Ajith:~$ stat file1.txt
  File: file1.txt
  Size: 19          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 46528         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ajithmutyala)   Gid: ( 1000/ajithmutyala)
Access: 2026-09-27 16:17:08.631194566 +0000
Modify: 2026-09-27 16:17:08.631194566 +0000
Change: 2026-09-27 16:17:08.631194566 +0000
 Birth: 2026-09-27 16:17:08.631194566 +0000
ajithmutyala@Ajith:~$ |
```

```
ajithmutyala@Ajith:~$ ln file1.txt hardlink.txt
ajithmutyala@Ajith:~$ ls -li file1.txt hardlink.txt
46528 file1.txt 46528 hardlink.txt
ajithmutyala@Ajith:~$ stat file1.txt
  File: file1.txt
  Size: 19          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 46528         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ajithmutyala)   Gid: ( 1000/ajithmutyala)
Access: 2026-09-27 16:17:08.631194566 +0000
Modify: 2026-09-27 16:18:59.103912111 +0000
Change: 2026-09-27 16:19:05.443394442 +0000
 Birth: 2026-09-27 16:17:08.631194566 +0000
ajithmutyala@Ajith:~$
```

```
ajithmutyala@Ajith:~$ echo "Hard link test" >> hardlink.txt
ajithmutyala@Ajith:~$ cat file1.txt
Hello Linux Inodes
ajithmutyala@Ajith:~$ echo "Hard link test" >> hardlink.txt
ajithmutyala@Ajith:~$ cat file1.txt
Hello Linux Inodes
ajithmutyala@Ajith:~$ cat hardlink.txt
Hard link test
ajithmutyala@Ajith:~$
```

```
ajithmutyala@Ajith:~$ ln -s file1.txt symlink.txt
ajithmutyala@Ajith:~$ ls -li file1.txt hardlink.txt symlink.txt
46528 file1.txt 46528 hardlink.txt 46441 symlink.txt
ajithmutyala@Ajith:~$ stat symlink.txt
  File: symlink.txt -> file1.txt
  Size: 9               Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48    Inode: 46441        Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/ajithmutyala)   Gid: ( 1000/ajithmuty
ala)
Access: 2026-09-27 16:23:05.535430391 +0000
Modify: 2026-09-27 16:22:59.560884444 +0000
Change: 2026-09-27 16:22:59.560884444 +0000
 Birth: 2026-09-27 16:22:59.560884444 +0000
ajithmutyala@Ajith:~$ stat -c '%i %h %F %n' symlink.txt
46441 1 symbolic link symlink.txt
ajithmutyala@Ajith:~$ |
```

```
ajithmutyala@Ajith:~$ find . -maxdepth 1 -printf '%i %p\n'
689 .
49425 ./config
49704 ./local
49345 ./cache
1230 ./bashrc
696 ./OSSP
46433 ./pipe.fifo
46441 ./symlink.txt
46427 ./HULK
49323 ./info.txt
49324 ./profile
46528 ./file1.txt
46528 ./hardlink.txt
46432 ./myfifo
49359 ./landscape
49361 ./motd_shown
46460 ./lowlevel.c.save
49374 ./bash_history
49410 ./process.c.save
49348 ./bash_logout
ajithmutyala@Ajith:~$
```

```
ajithmutyala@Ajith:~$ find . -inum "$(ls -li file1.txt | awk '{print $1}')"
./file1.txt
./hardlink.txt
ajithmutyala@Ajith:~$
```

```

ajithmutyala@Ajith:~$ rm hardlink.txt
stat file1.txt
rm symlink.txt
ls -l file1.txt
rm: cannot remove 'hardlink.txt': No such file or directory
  File: file1.txt
  Size: 19          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 46528         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ajithmutyala)   Gid: ( 1000/ajithmuty
ala)
Access: 2026-09-27 16:20:48.981321283 +0000
Modify: 2026-09-27 16:27:24.612365175 +0000
Change: 2026-09-27 16:30:37.716604964 +0000
Birth: 2026-09-27 16:17:08.631194566 +0000
rm: cannot remove 'symlink.txt': No such file or directory
-rw-r--r-- 1 ajithmutyala ajithmutyala 19 Sep 27 16:27 file1.txt

```

Report: The inode structure was investigated using `ls -li`, `stat`, and `find` commands. Hard links shared the same inode, while symbolic links had separate inodes. The experiment demonstrated how link creation and deletion affect inode allocation.

2. Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

int main() {
    int src, dest;
    struct stat st;
    char *source, *output;
    src = open("input.txt", O_RDONLY);
    if (src < 0) {
        perror("Source file error");
        return 1;
    }
    fstat(src, &st);
    size_t size = st.st_size;
    if (size == 0) {
        printf("Empty file\n");
        close(src);
        return 0;
    }
    dest = open("output.txt",
               O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (dest < 0) {
        perror("Destination error");
        close(src);
        return 1;
    }
    ftruncate(dest, size);
    source = mmap(NULL, size, PROT_READ,
                  MAP_PRIVATE, src, 0);
    output = mmap(NULL, size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, dest, 0);
    if (source == MAP_FAILED || output == MAP_FAILED) {
```

```
ajithmutyala@Ajith:~$ nano mmap_demo.c
ajithmutyala@Ajith:~$ echo "Hello Linux Memory Mapping" > input.txt
ajithmutyala@Ajith:~$ cat input.txt
Hello Linux Memory Mapping
ajithmutyala@Ajith:~$ gcc mmap_demo.c -o mmap_demo
ajithmutyala@Ajith:~$ ./mmap_demo
File copied successfully using mmap()
File size: 27 bytes
Copy time: 0.000218 seconds
```

```
GNU nano 8.7.1 traditional.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

int main() {
    int src, dest;
    char buffer[4096];
    ssize_t n;

    src = open("input.txt", O_RDONLY);
    dest = open("output2.txt",
               O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (src < 0 || dest < 0) {
        perror("File error");
        return 1;
    }

    clock_t start = clock();

    while ((n = read(src, buffer, sizeof(buffer))) > 0) {
        write(dest, buffer, n);
    }

    clock_t end = clock();

    printf("File copied using read() and write()\n");
    printf("Copy time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    close(src);
    close(dest);

    return 0;
}
```

```
ajithmutyala@Ajith:~$ nano traditional.c
ajithmutyala@Ajith:~$ echo "Hello Linux Memory Mapping" > input.txt
ajithmutyala@Ajith:~$ gcc traditional.c -o traditional
ajithmutyala@Ajith:~$ ./traditional
File copied using read() and write()
Copy time: 0.000021 seconds
```

```
ajithmutyala@Ajith:~$ time ./traditional
File copied using read() and write()
Copy time: 0.000307 seconds

real    0m0.018s
user    0m0.000s
sys     0m0.002s
ajithmutyala@Ajith:~$ time ./mmap_demo
time ./traditional
-bash: ./mmap_demo: No such file or directory

real    0m0.001s
user    0m0.000s
sys     0m0.001s
```

Report: The file was successfully read and written using mmap() and traditional read() and write() system calls. Both methods were verified using cmp, and their execution times were compared using time.